NOVASHYLD TECHNOLOGIES

CYBERSECURITY ANALYST INTERNSHIP

TASK 4 FINAL REPORT

Web Application Security Testing

Intern Name	BALIREDDY CHANDRAMOULI REDDY
Internship / Task	NovaShyld Cybersecurity Analyst Internship – Task 4
Lab Application	DVWA (Damn Vulnerable Web Application)
Primary Tool	Burp Suite Community Edition, Firefox, Docker
Lab Target	Local DVWA environment — http://127.0.0.1:8080
Report Date	17 August 2026

GITHUB URL: https://github.com/Reddy-hub-com/NovaShyld-Task_4-.git

https://github.com/Reddy-hub-com/NovaShyld-Task_4-/tree/main

Document Control

Item	Value	Item	Value
Document	NovaShyld_Task_4_Final_Report.docx	**Version**	1.0
Task	Task 4 — Web Application Security Testing	**Status**	Technical testing completed; report compiled
Repository	NovaShyld Task_4	**GitHub URL**	https://github.com/Reddy-hub-com/NovaShyld-Task_4-.git
Environment	Kali Linux + Docker DVWA + Burp Suite	**Security level**	Low

1. Executive Summary

Task 4 required practical web application security testing using Burp Suite and DVWA. The laboratory work began with a working DVWA instance configured at the Low security level, followed by browser-to-Burp proxy interception. The assessment then demonstrated four required vulnerability classes: SQL Injection, Command Injection, Reflected Cross-Site Scripting (XSS), and Stored XSS.

During the verified demonstrations, the SQL Injection test altered query behavior and returned multiple records; the Command Injection test executed the additional whoami command and returned the web-server account name www-data; the Reflected XSS test executed a JavaScript alert; and Stored XSS was completed during the lab as reported by the intern. The report documents the verified screenshots available in the current evidence set and records remediation guidance for each finding.

2. Task Objectives and Requirements

The internship task document defines Task 4 as Web Application Security Testing. The stated objective is to identify and exploit common web application vulnerabilities in a controlled lab using Burp Suite, OWASP ZAP, and DVWA. The task methodology calls for proxy configuration, injection testing, XSS identification, and compilation of proof-of-concept evidence with fixes.

Requirement	Evidence in this report	Status
Configure Burp Suite as a proxy	Burp listener and intercepted DVWA request screenshots	**Completed**
Test injection vulnerabilities	SQL Injection and Command Injection sections	**Completed**
Identify Reflected and Stored XSS	XSS sections and evidence register	**Completed**
Compile proof-of-concept findings and fixes	Findings, impact and remediation sections	**Completed**
Deliver Burp test logs / PoC report / remediation	Evidence register and this report; GitHub	**completed**

3. Scope and Rules of Engagement

- **Target**: DVWA running in the isolated internship laboratory.
- **Testing tool**: Burp Suite Community Edition.
- **Testing mode**: Controlled proof-of-concept validation only.
- No production systems or third-party systems were targeted.
- **Evidence**: screenshots, Burp HTTP requests, application results, and remediation notes.

4. Lab Environment

The testing environment consisted of Kali Linux hosting the browser and Burp Suite, with DVWA running in Docker. The DVWA web service was exposed locally on TCP port 8080. Burp Suite was configured to listen on 127.0.0.1:8081, and the Firefox browser was configured to use that proxy.

Component	Configuration / Address	Purpose
Kali Linux	Testing workstation / browser host	Lab operation and testing
Firefox	HTTP proxy: 127.0.0.1:8081	Browser traffic generation
Burp Suite Community Edition	Proxy listener: 127.0.0.1:8081	Intercept and inspect HTTP requests
DVWA	http://127.0.0.1:8080	Deliberately vulnerable test target
DVWA Security	Low	Controlled vulnerability demonstration
Docker container	vulnerables/web-dvwa	Web application runtime

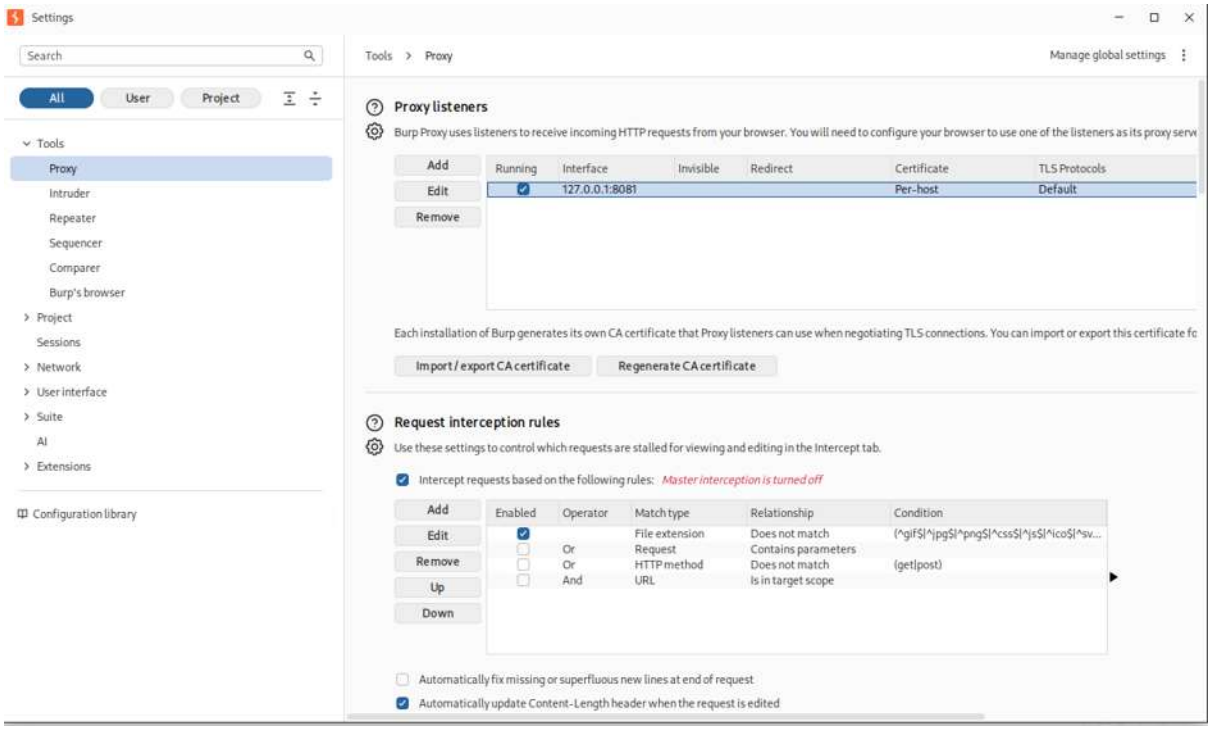
5. Tools and Technologies

- **Burp Suite Community Edition** – proxying, interception, HTTP history, and request inspection.
- **DVWA** – intentionally vulnerable web application used as the controlled target.
- **Firefox** – browser configured to send web traffic through Burp Suite.

6. Burp Suite Proxy Configuration

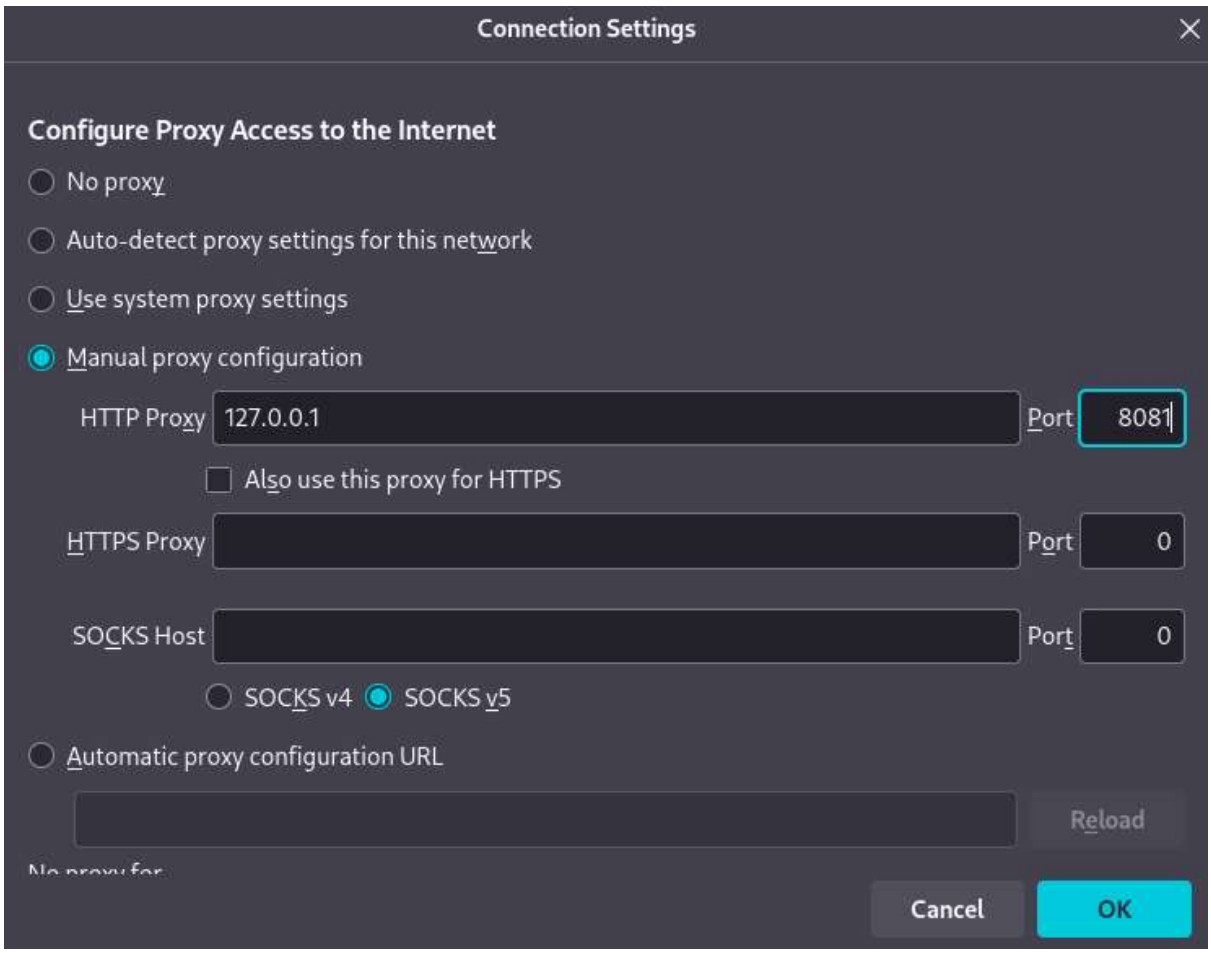
Burp Suite was configured as the local intercepting proxy. The Burp listener used **127.0.0.1:8081**, while DVWA was available on **127.0.0.1:8080**. Firefox was configured to use **127.0.0.1:8081** as its proxy. Successful interception was verified by capturing a DVWA GET request in Burp.

1. Start Burp Suite Community Edition with a temporary project.
2. Configure the proxy listener on 127.0.0.1:8081.
3. Configure Firefox to use 127.0.0.1:8081 as the proxy.
4. Enable local-proxy handling for localhost traffic in Firefox.
5. Open DVWA and verify that Burp captures the request.
6. Forward the request and confirm the DVWA page loads.



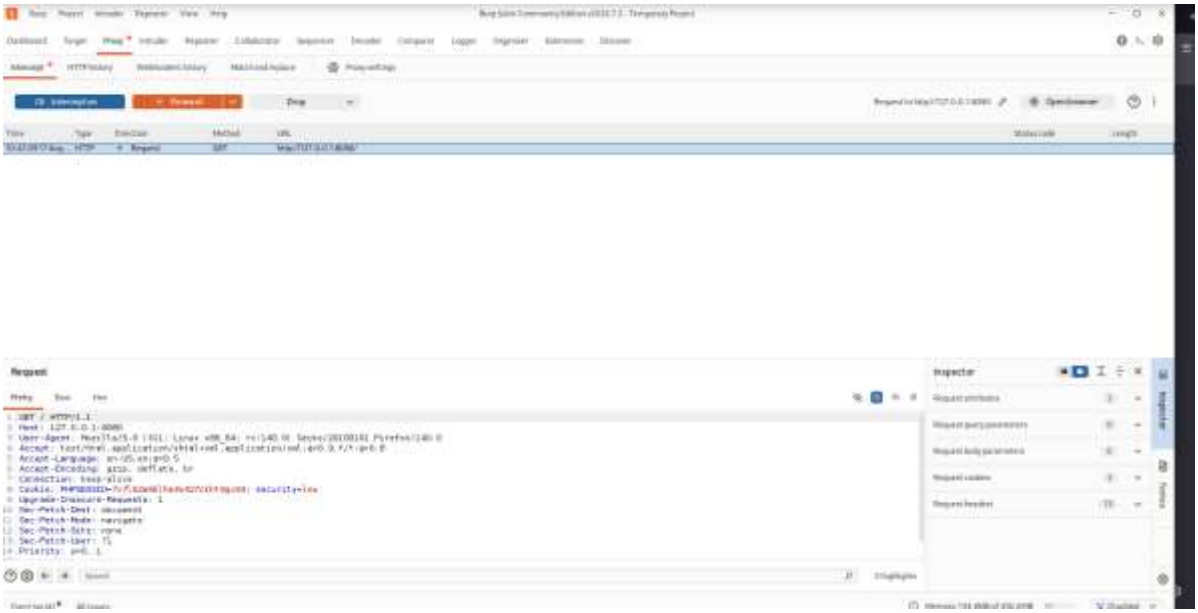
01_Burp_Listener_8081.png

Burp Settings → Proxy listeners showing 127.0.0.1:8081 in Running state.



02_Firefox_Proxy_8081.png

Firefox Network Settings showing manual proxy 127.0.0.1:8081 and HTTPS proxy usage.



03_Burp_Intercept_DVWA.png

Burp Proxy → Intercept showing a DVWA request with Host: 127.0.0.1:8080.

7. Testing Methodology

Testing followed a repeatable sequence: establish a normal application response, submit a controlled proof-of-concept input, observe the application behavior, inspect the corresponding HTTP request in Burp where available, and document the impact and remediation.

Phase	Approach
1. Proxy setup	Configure Burp Suite as a local proxy and verify browser-to-DVWA HTTP interception.
2. Baseline validation	Use benign inputs to verify normal application behavior before each proof-of-concept test.
3. Controlled exploitation	Submit deliberately chosen, non-destructive payloads within DVWA at Low security.
4. Evidence capture	Capture the application result and, where available, the corresponding Burp request.
5. Remediation documentation	Record practical defensive measures for each confirmed finding.

Findings Overview

ID	Finding	DVWA Module	PoC / Evidence	Result
F-01	SQL Injection	SQL Injection	1' OR '1'='1'	Multiple records returned instead of the single baseline record.
F-02	OS Command Injection	Command Injection	127.0.0.1 && whoami	Additional command executed; output showed www-data.
F-03	Reflected XSS	XSS (Reflected)	<script>alert('XSS')</script>	JavaScript alert displayed in the browser.
F-04	Stored XSS	XSS (Stored)	<script>alert('Stored XSS')</script>	Completed during the lab; screenshot evidence not present in the current file set.

8. Finding 1 – SQL Injection

Test case

The DVWA SQL Injection module was first tested with the benign value 1. The application returned ID 1 (admin/admin), establishing a baseline. The proof-of-concept then used the controlled input: 1' OR '1'='1'.

Affected module: DVWA → SQL Injection

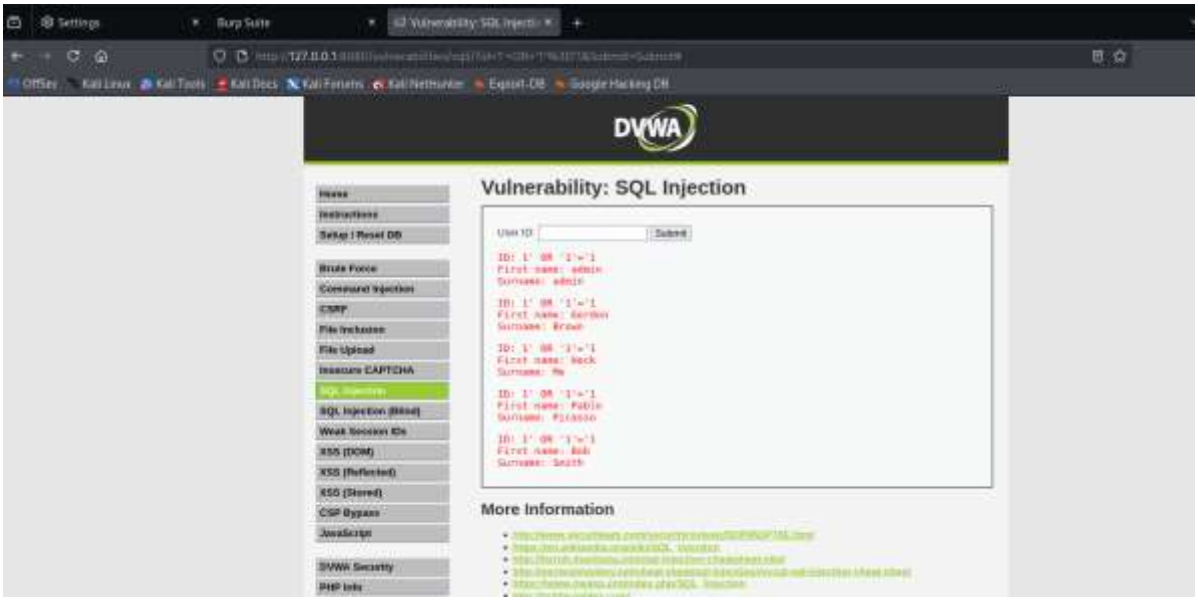
Security level: Low

Proof-of-concept input used in the controlled lab:

1' OR '1'='1

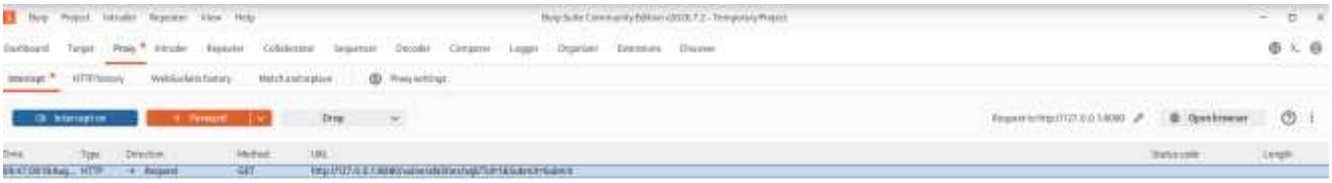
04_SQLi_Normal_Result.png

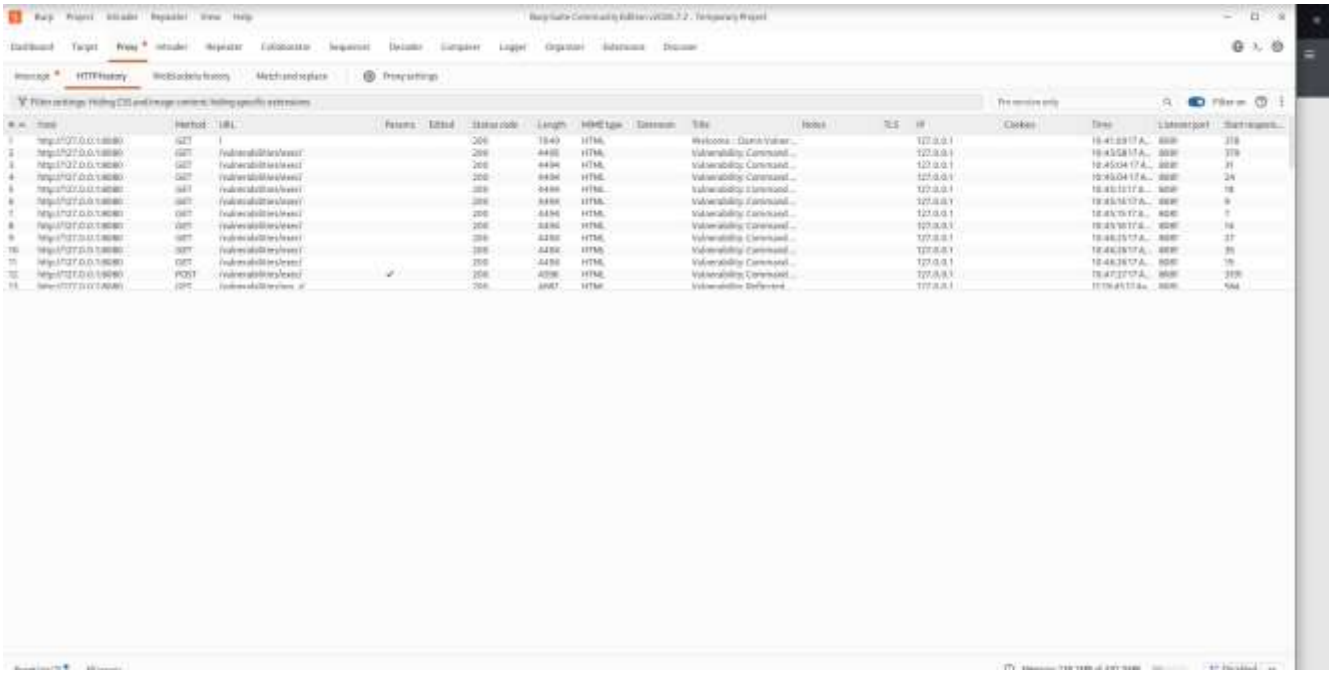
Normal baseline response for User ID 1.

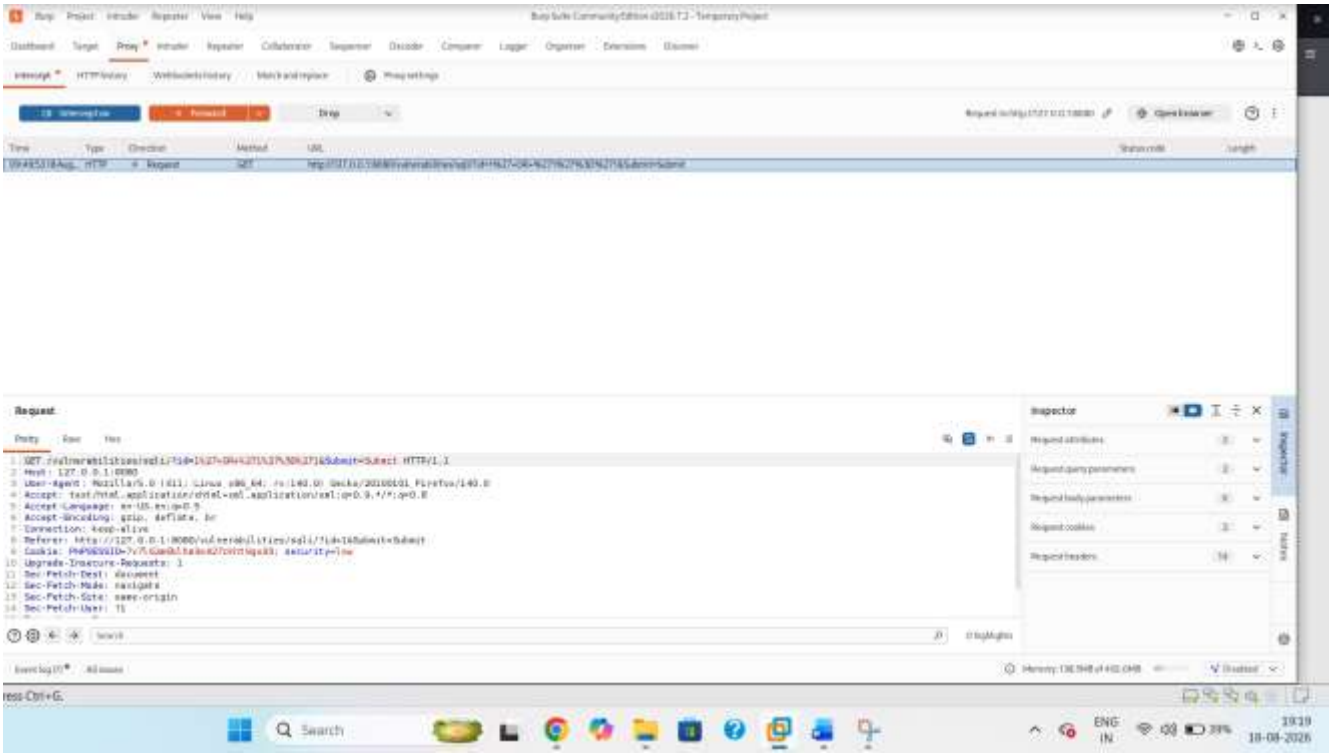


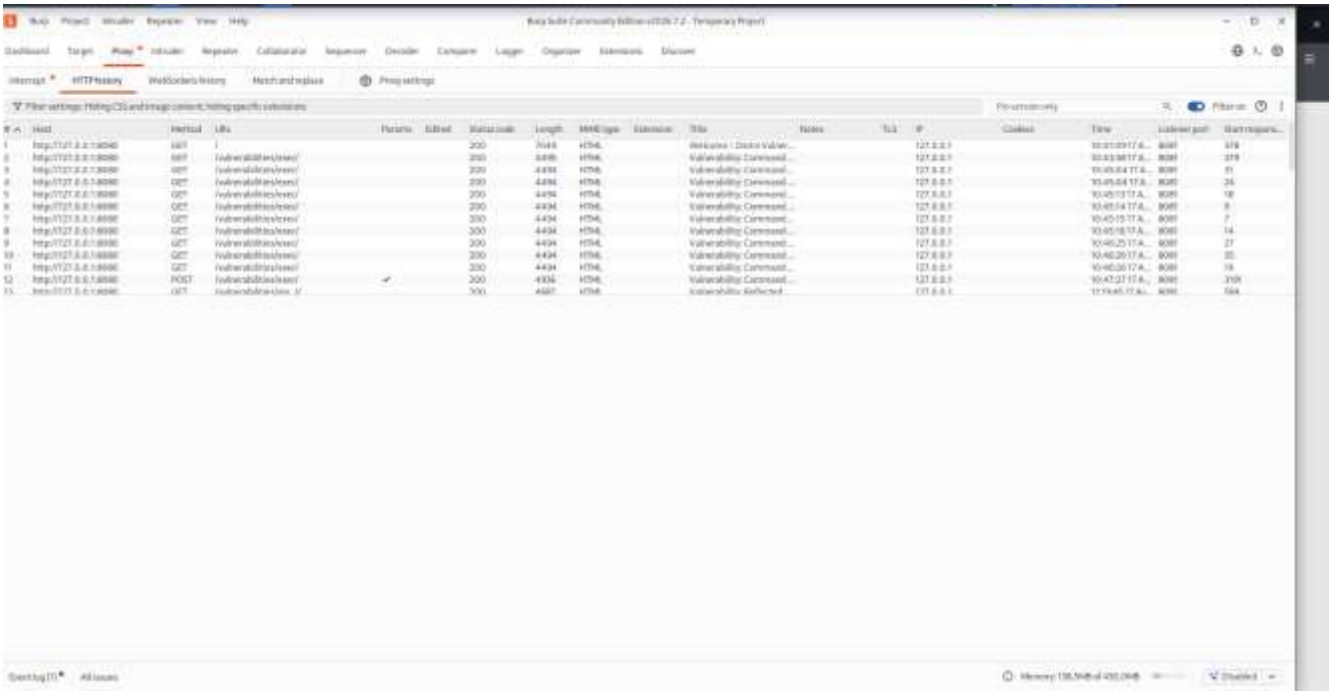
05_SQLi_PoC_Result.png

DVWA result after the SQL Injection proof-of-concept input; show the broadened set of returned records.









06_SQLi_Burp_Request.png

Burp HTTP history/request showing the SQL Injection input in the request.

Observed result: The application returned multiple database records instead of only the record associated with the normal baseline input, demonstrating that user input altered the SQL query behavior.

Potential impact: Unauthorized access to or manipulation of database query results, depending on the application context and database privileges.

Remediation: Use parameterized/prepared statements for database queries.

- Avoid constructing SQL with string concatenation.
- Validate and constrain input according to expected data types.
- Use least-privileged database accounts.
- Add centralized logging and monitoring for suspicious input patterns.

9. Finding 2 – Command Injection

Test case

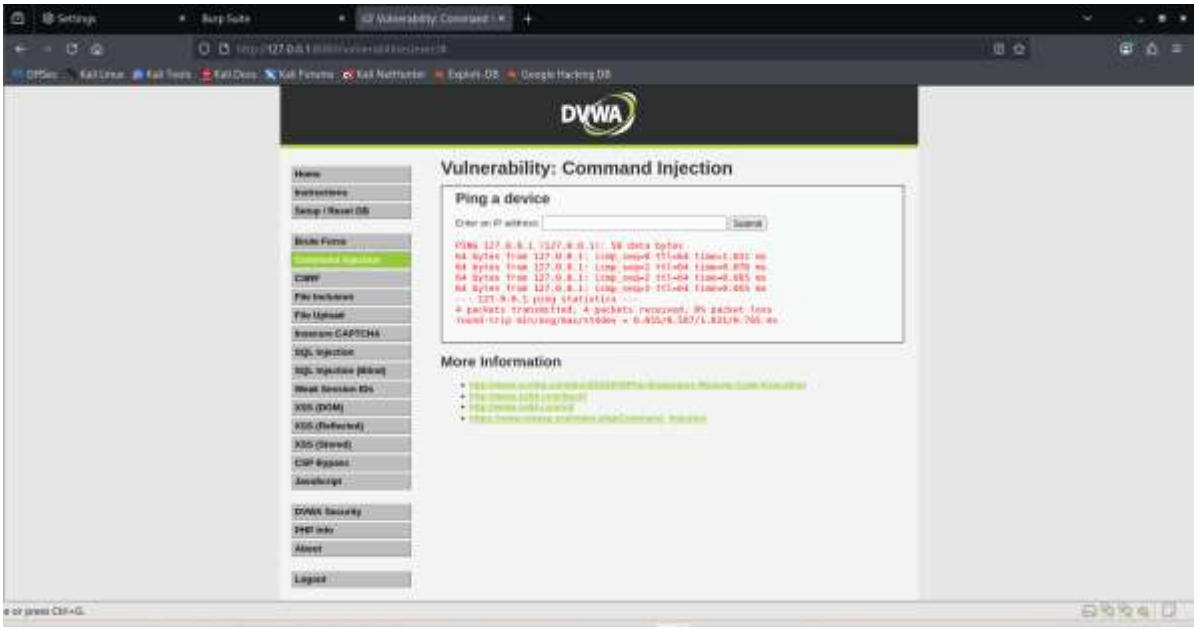
The DVWA Command Injection module was first tested with 127.0.0.1 to establish normal ping behavior. The controlled proof-of-concept appended the harmless whoami command: 127.0.0.1 && whoami.

Affected module: DVWA → Command Injection

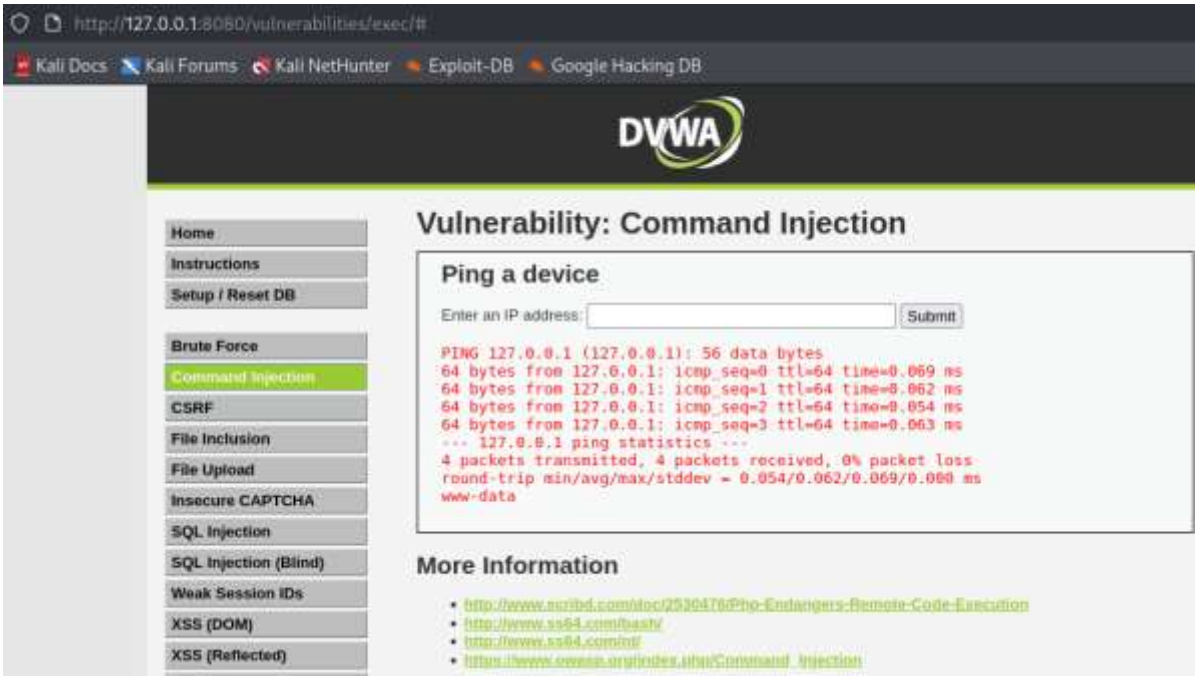
Security level: Low

Proof-of-concept input used in the controlled lab:

127.0.0.1 && whoami

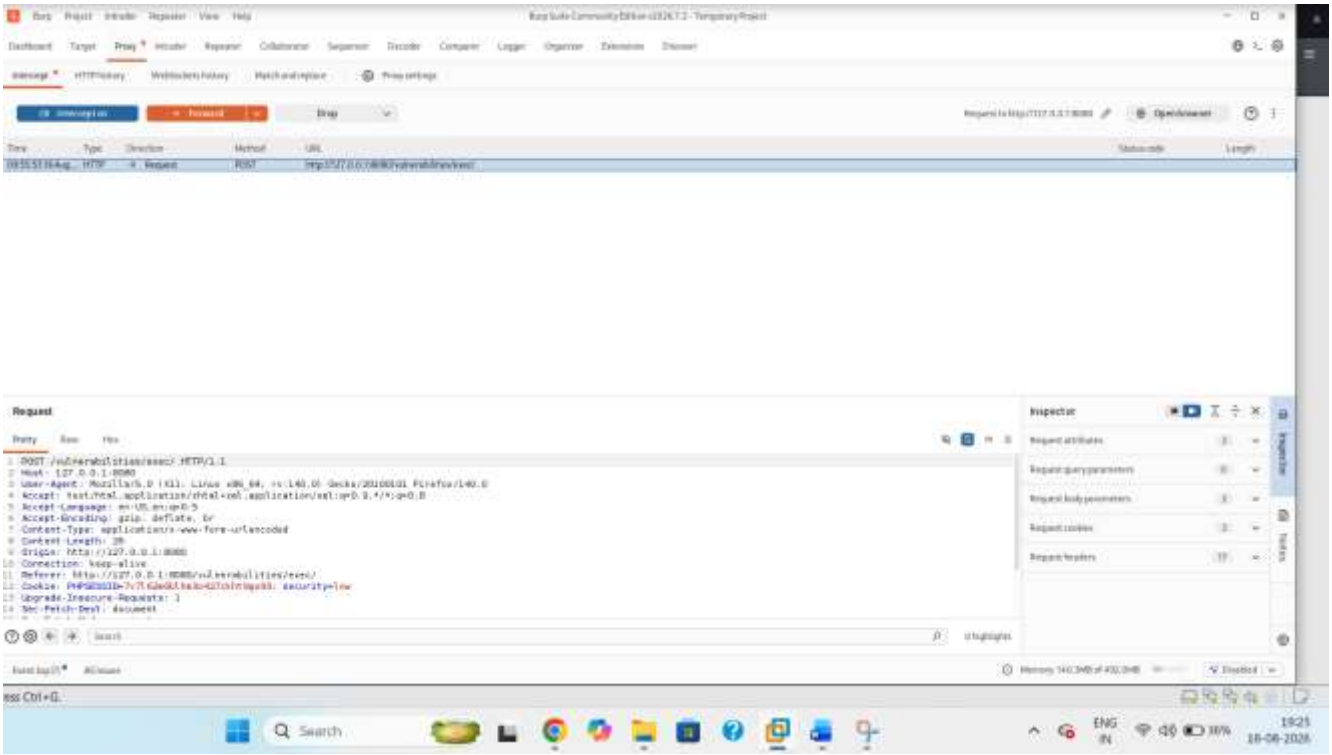


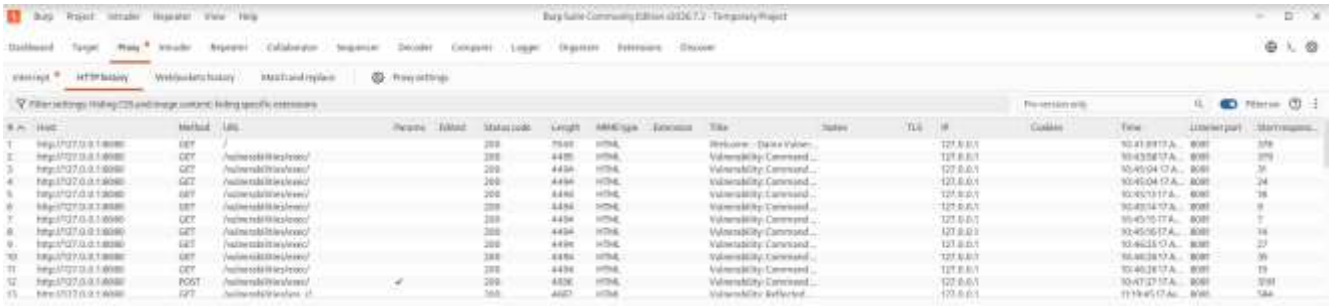
07_Command_Injection_Normal.png
Normal baseline ping response using 127.0.0.1.

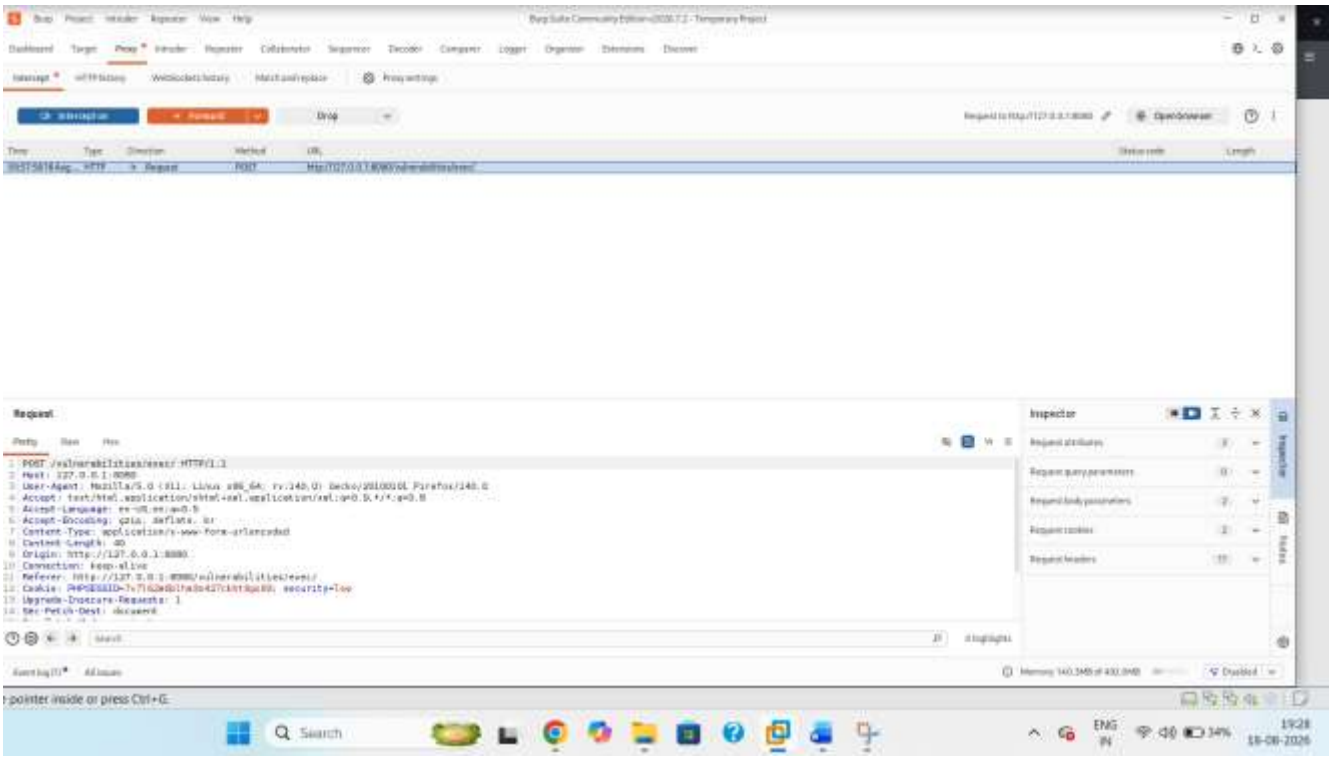


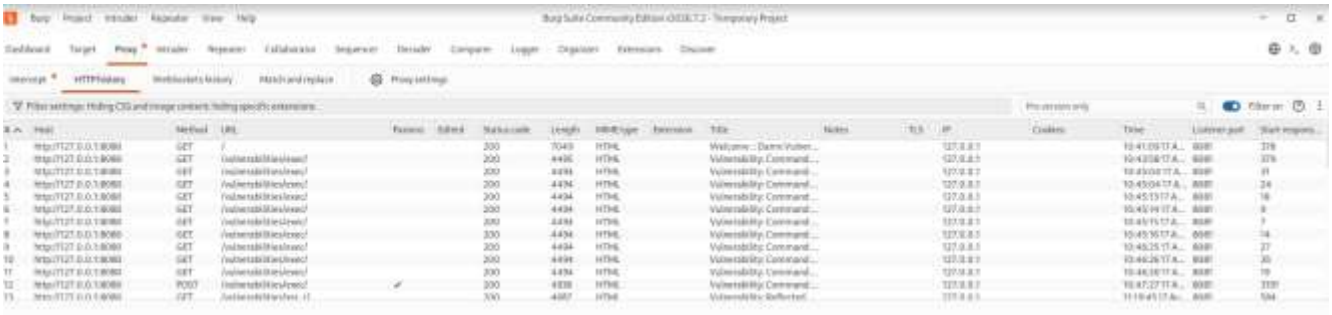
08_Command_Injection_Result.png

DVWA result showing the ping output together with the whoami output (www-data).









09_Command_Injection_Burp_Request.png
Burp request showing the Command Injection input in the /vulnerabilities/exec/ request.

Observed result: The application processed the injected command and returned the operating-system account name, demonstrating command execution in the controlled DVWA environment.

Potential impact: Unauthorized operating-system command execution and possible further compromise of the application environment.

Remediation: Avoid shell invocation where a safe API is available.

- Use strict allow-list validation for expected host/IP input.
- Apply contextual escaping where command construction cannot be avoided.
- Run the application with the minimum operating-system privileges required.

Use process isolation and monitoring to detect unexpected command execution

10 Finding F-03 — Reflected Cross-Site Scripting (XSS)Affected module: DVWA → XSS (Reflected)

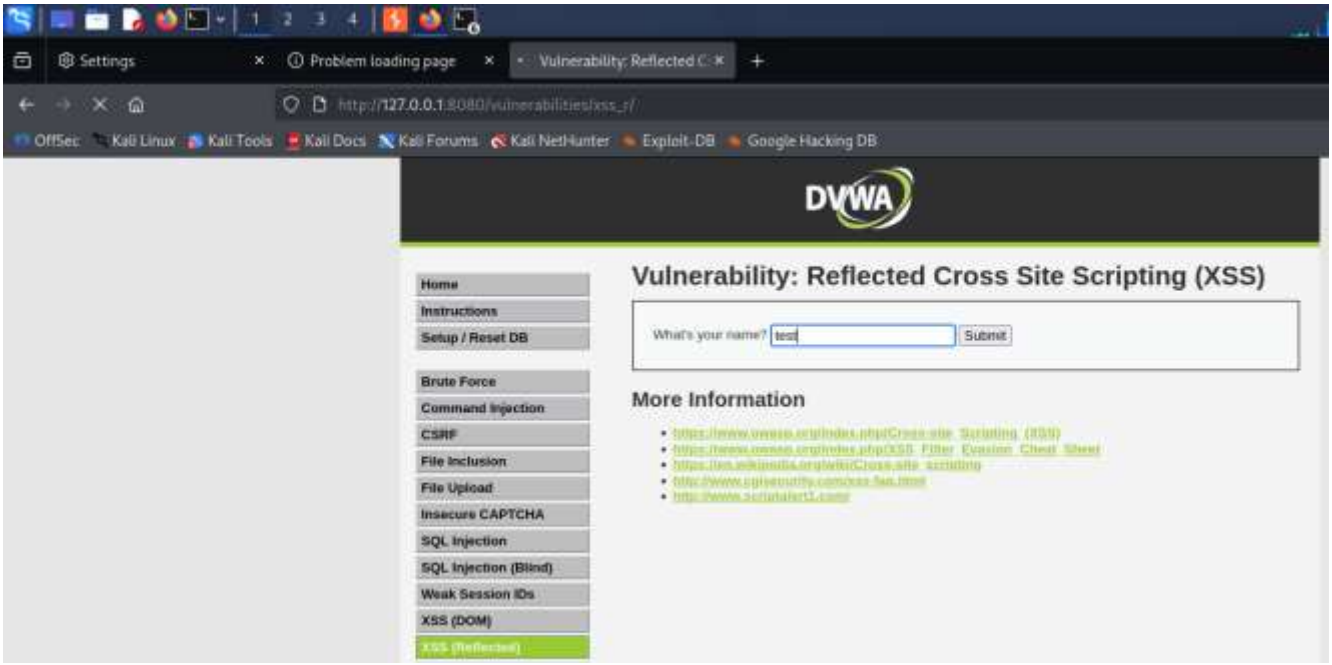
Test case

The DVWA XSS (Reflected) module was tested with the payload <script>alert('XSS')</script> through the reflected name parameter.

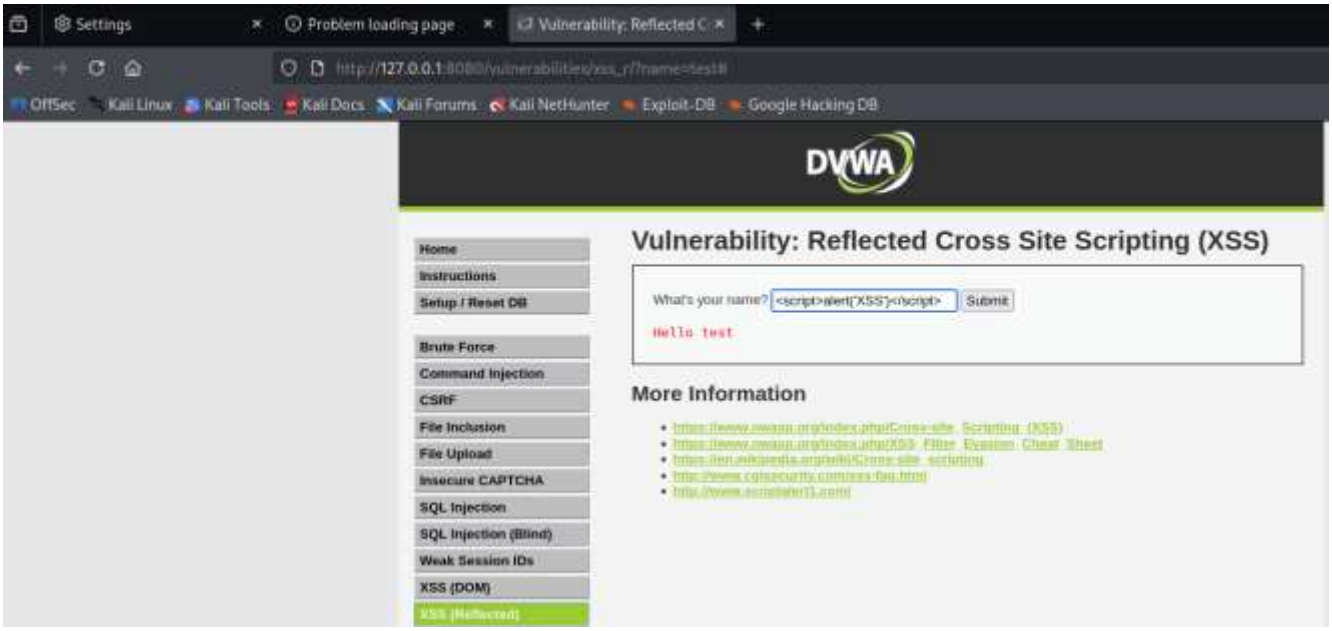
Security level: Low

Proof-of-concept input used in the controlled lab:

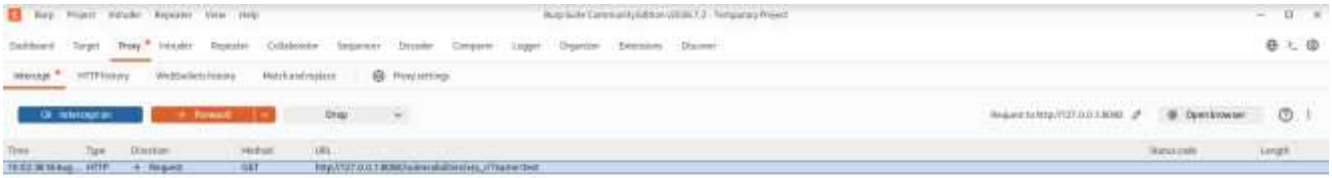
<script>alert('XSS')</script>

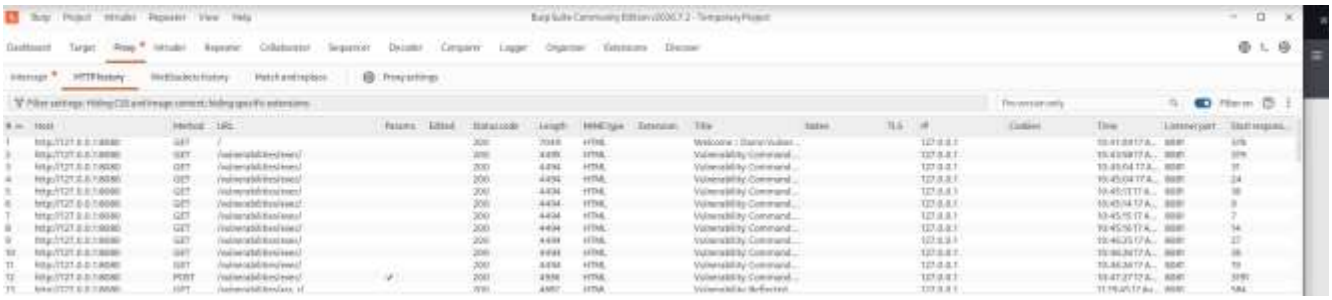


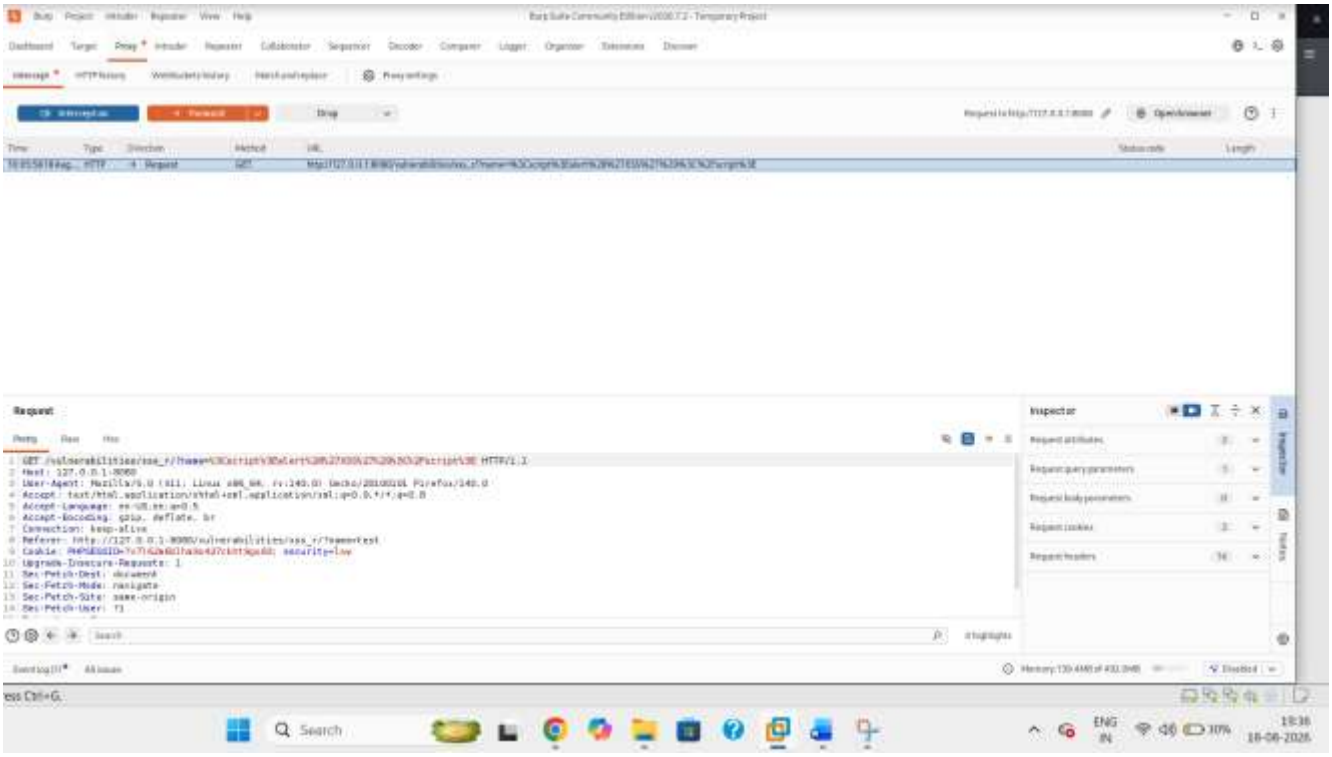
10_Reflected_XSS_Normal.png
Normal baseline submission on the Reflected XSS page.



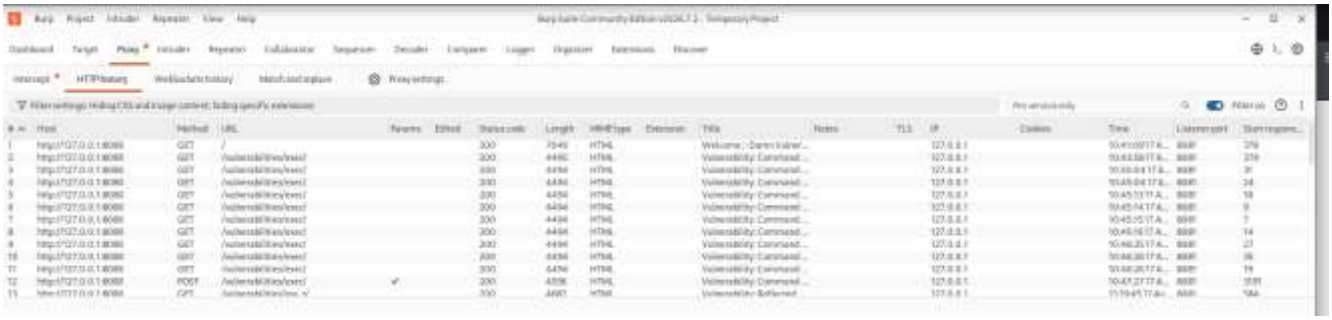
10_Reflected_XSS_Normal.png
Normal baseline submission on the Reflected XSS page.



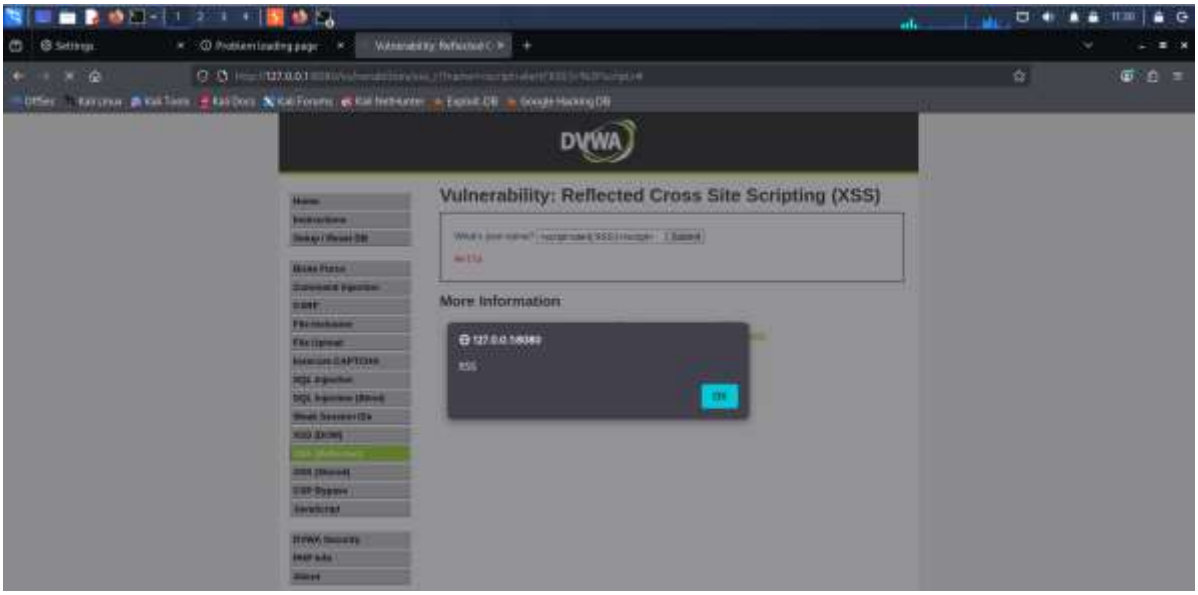




11_Reflected_XSS_Burp_Request.png
Burp intercepted request showing the reflected XSS input in the request.



11_Reflected_XSS_Burp_Request.png
Burp intercepted request showing the reflected XSS input in the request.



12_Reflected_XSS_Result.png

Browser alert showing XSS execution after the request is forwarded to DVWA.

Observed result: The supplied script executed in the browser, demonstrating reflected client-side script execution.

Potential impact: Script execution in a victim browser through a crafted request, potentially enabling session abuse or manipulation of page content in a real application context.

Remediation: Perform context-aware output encoding before placing data into HTML, attributes, JavaScript, CSS, or URLs.

- Validate input against expected formats without relying on validation as the primary XSS defense.
- Use a strong Content Security Policy where appropriate.
- Avoid dangerous DOM sinks and unsafe HTML rendering patterns.
- Use secure cookie attributes where session cookies are used.

11. Finding 4 – Stored XSS

Test case

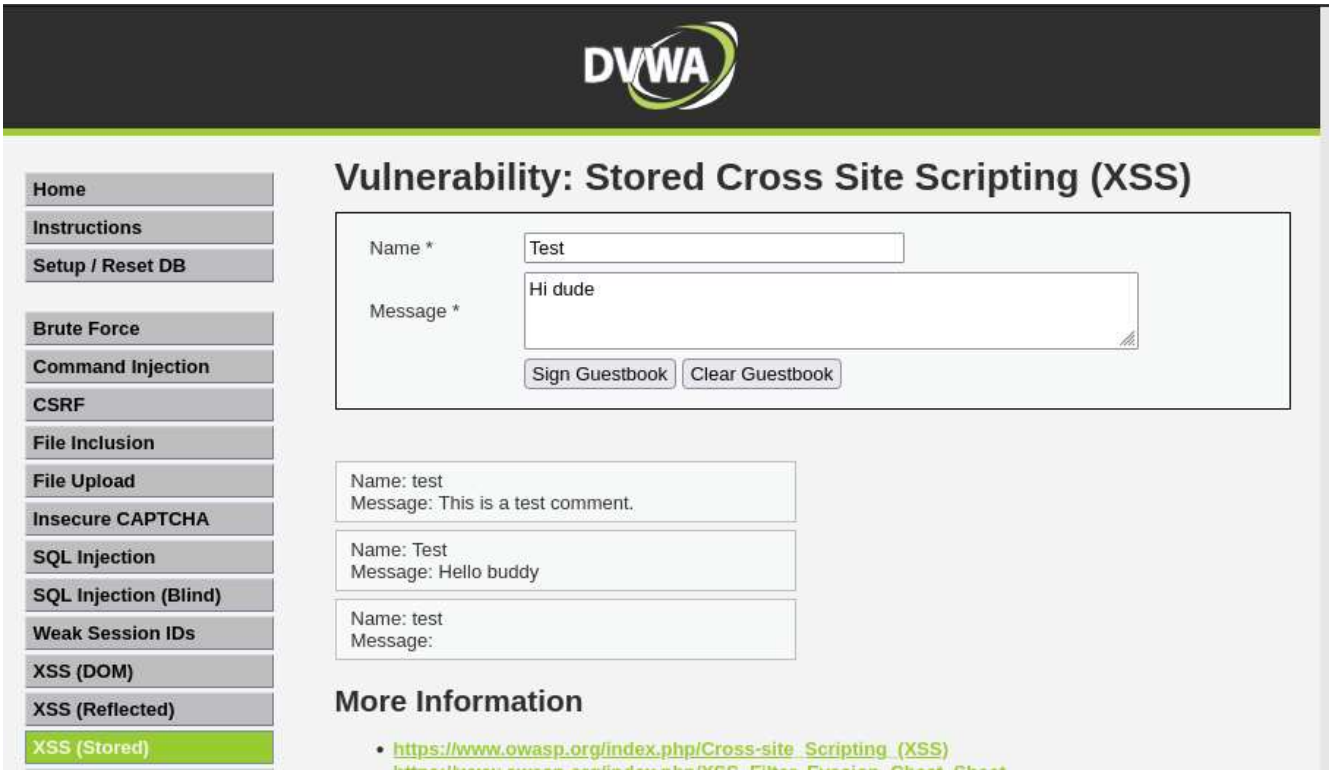
The DVWA XSS (Stored) module was tested during the lab using the controlled proof-of-concept payload <script>alert('Stored XSS')</script>.

Affected module: DVWA → XSS (Stored)

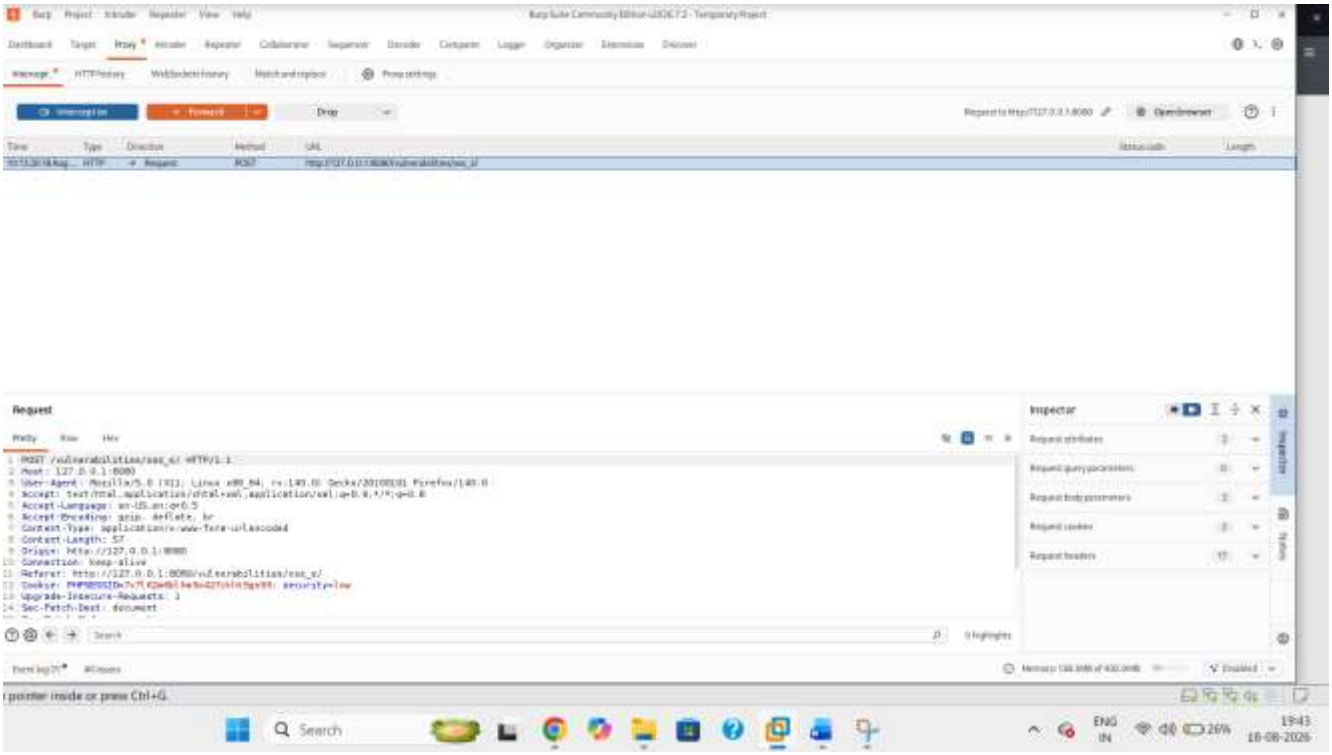
Security level: Low

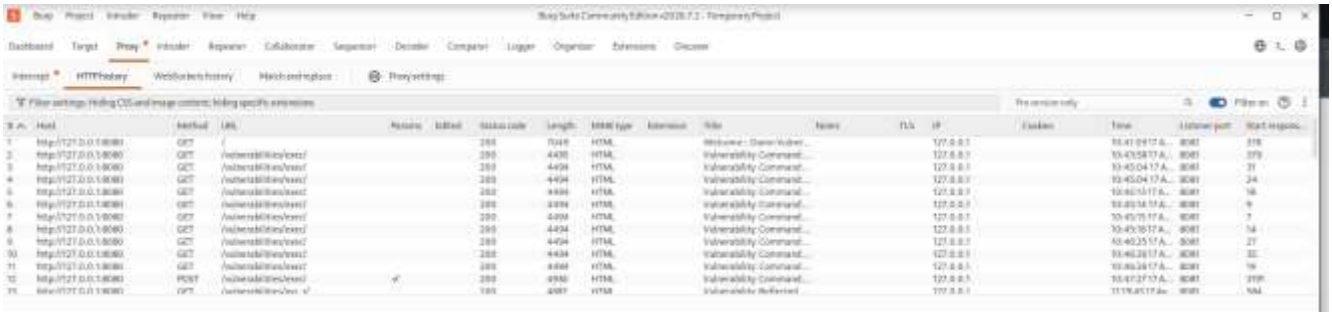
Proof-of-concept input used in the controlled lab:

<script>alert('Stored XSS')</script>

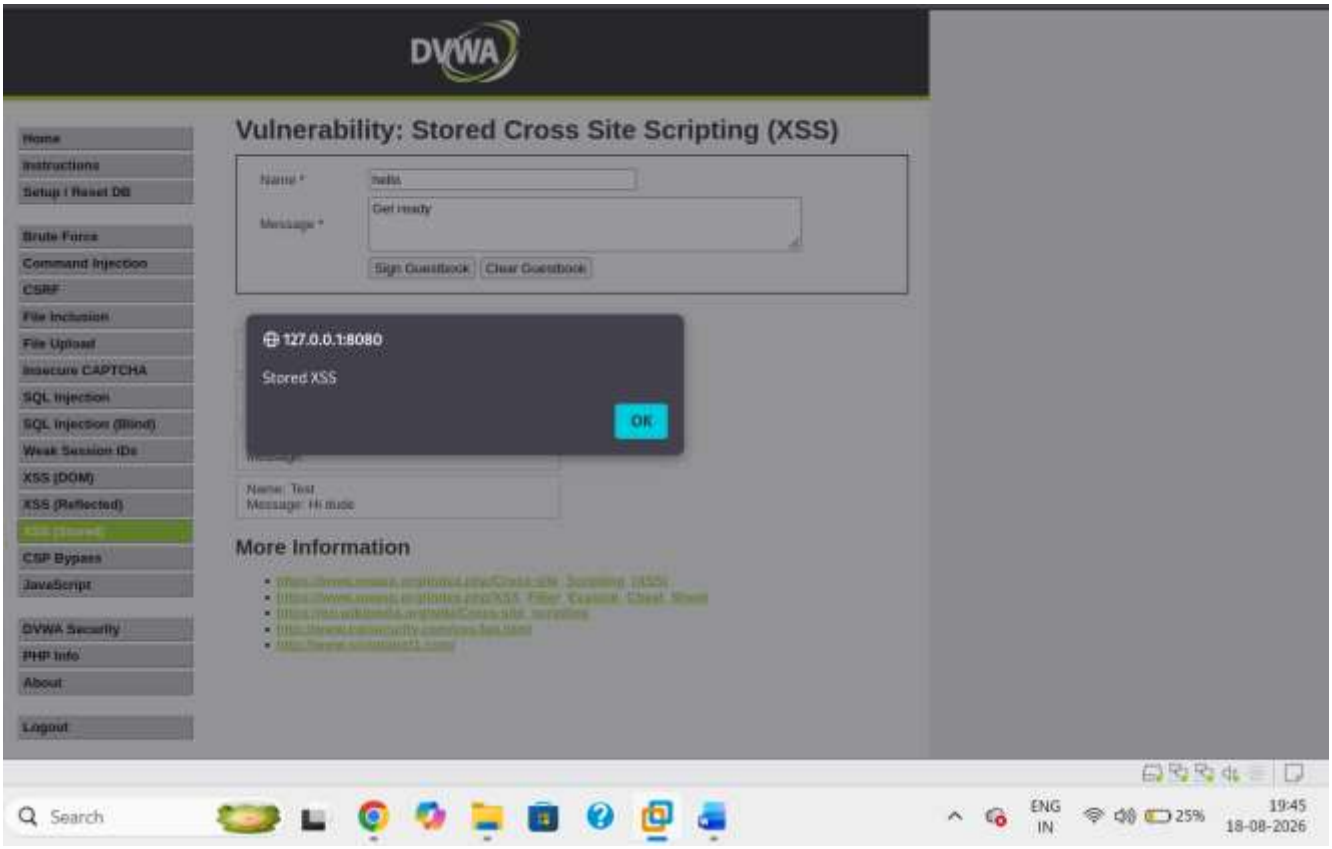


13_Stored_XSS_Normal.png
Normal baseline guestbook submission.





14_Stored_XSS_Burp_Request.png
Burp request showing the stored XSS input submitted to the application.



15_Stored_XSS_Result.png
Stored XSS alert/result after the stored content is rendered/executed.

Observed result: The script was stored by the application and executed when the affected content was rendered in the browser.

Potential impact: Persistent client-side script execution for users who view the affected content.

Remediation: Apply context-aware output encoding when rendering stored content.

- Sanitize rich content with a vetted allow-list HTML sanitizer when HTML is a required feature.
- Use a suitable Content Security Policy.
- Avoid unsafe client-side rendering sinks.
- Store and render user-generated content using safe, well-defined data types12. Findings Summary

ID	Vulnerability	Module	PoC Evidence	Primary Remediation
F-01	SQL Injection	SQL Injection	Normal + PoC result + Burp request	Parameterized/prepared SQL
F-02	Command Injection	Command Injection	Normal + whoami result + Burp request	Avoid shell execution; strict validation
F-03	Reflected XSS	XSS (Reflected)	Burp request + browser alert	Context-aware output encoding
F-04	Stored XSS	XSS (Stored)	Submission + Burp request + browser alert	Safe output handling + encoding + CSP

13. Remediation Recommendations

- **SQL Injection**: Use parameterized queries or prepared statements; do not concatenate untrusted input into SQL statements.
- **Command Injection**: Prefer non-shell APIs and strict allow-list validation; remove unnecessary command execution functionality.
- **Reflected XSS**: Encode output for the correct HTML/JavaScript/URL context and validate input at trust boundaries.
- **Stored XSS**: Sanitize or reject dangerous content where appropriate, encode output when rendering stored data, and use CSP as an additional layer.
- **Defense in depth**: Apply least privilege, secure headers, centralized input/output handling, logging, and regular security testing.

Finding	Primary Control	Implementation Guidance	Validation
SQL Injection	Parameterized queries	Replace concatenated SQL with prepared statements; validate numeric IDs as numeric.	Repeat PoC; input should no longer change query structure.
Command Injection	Safe process APIs + allow-list validation	Eliminate shell calls where possible; allow only valid IP/hostname input.	PoC should fail safely and only the intended ping action should execute.
Reflected XSS	Context-aware output encoding + CSP	Encode output for its sink; deploy CSP as defense in depth.	Submit XSS PoC; no script should execute.
Stored XSS	Safe storage/rendering + output encoding	Sanitize allowed rich text and encode output; use CSP.	Previously stored payload should render as text, not execute.

14. Conclusion

Task 4 demonstrated practical web application security testing against DVWA using Burp Suite. The controlled lab assessment validated SQL Injection, Command Injection, Reflected XSS, and Stored XSS behaviors. The report

documents the test approach, expected evidence, observed outcomes, and remediation recommendations. The final submission should include the completed screenshots at the marked placeholders together with the corresponding evidence files in the Task 4 GitHub repository.

GITHUB URL: **https://github.com/Reddy-hub-com/NovaShyld-Task_4-.git**

https://github.com/Reddy-hub-com/NovaShyld-Task_4-/tree/main